Ticket #10 – central logs on stor01 · walkthrough

Project 2 – Acme Analytics Linux platform · Phase 5, Step 5.4 · Day 17 (11 Nov 2026) · every step: why → command → screenshot → the line that matters

Phase 5 · Step 5.4 – Central logs on stor01 (rsyslog)

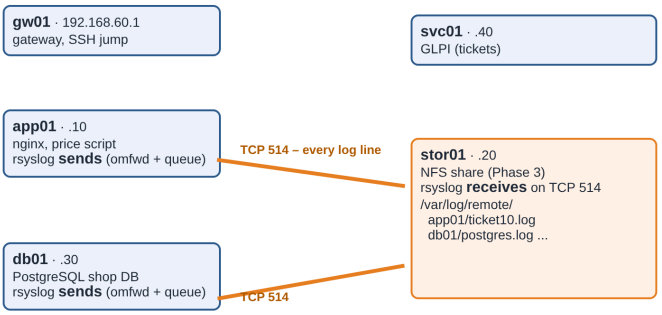
Project 2 – Acme Analytics Linux platform · Day 17 · 11 Nov 2026

GLPI · Ticket #10 · New

Title	**Logs are spread over every server – after a freeze we cannot read them**
Requester	Sara (IT lead), Acme Analytics
Category	Linux › Logging
Priority	High
Servers	app01, db01 (send) · stor01 (keep)

"On Black Friday night the price script failed on **app01** (ticket #8), and when app01 froze (ticket #9) we had to wait until it came back to read its log. For the shop DB we log in to **db01** separately. Please collect the logs of all servers in **one place**, so we can still read them when a server is frozen or broken – and an attacker on one server cannot delete the copy."

Real-world layout after Step 5.4



Network VMnet2 192.168.60.0/24 · each server still keeps its own local log too (journalctl) – stor01 has the **extra copy**.

What earlier phases give this step

Phase 1–2: VMs, static IPs, /etc/hosts, SSH keys from gw01
Phase 3: stor01 is the storage server (good place for the log copy)
Phase 5.2: **logger** + price script write tagged log lines
Day 16: db01 + PostgreSQL – a second server to send logs

New words

rsyslog – the program on Ubuntu that writes log lines to files and can send them over the network
port 514 – the "door" for log messages
imtcp – rsyslog part that *receives* · **omfwd** – part that *sends*
queue – a waiting line on disk: if stor01 is down, lines wait and are sent later

Plan

1. **A · stor01:** start it, add db01 to /etc/hosts, set rsyslog to receive on 514
2. **B · app01 + db01:** send every log line to stor01, with a queue
3. **C · Test:** logger on app01/db01 → line appears on stor01
4. **D · Failure test:** stop stor01's rsyslog, log, start it → waiting lines arrive
5. **E · Close ticket #10** with screenshots

The story in 4 lines. Ticket #8: the price script got its own log lines – only on app01. Ticket #9: the answer was in logs spread over app01 and db01, and frozen app01 could not be read. Logs that live only on their own server are lost when the server is frozen or gone, or when an attacker, an insider or a mistake deletes them. **Ticket #10:** every server keeps its own log **and** sends a copy to stor01.

Server	Role	File	What it does
stor01 (.20)	receiver	/etc/rsyslog.d/10-receive.conf	listens on TCP 514, saves /var/log/remote/<server>/<program>.log
app01 (.10), db01 (.30)	senders	/etc/rsyslog.d/90-forward.conf	sends every line (*.*) to 192.168.60.20:514 over TCP, with a queue
db01	+ PostgreSQL	postgresql.conf	log_destination = 'stderr,syslog' – the DB log also goes to rsyslog

How to read: orange number = a step · yellow box in a screenshot = the line that matters · orange box = an empty answer that proves something.

Ticket #10 – how central logging works

Project 2 – Acme Analytics Linux platform · Step 5.4 · rsyslog · my understanding, corrected · final

1 · Why

Tickets #8 and #9: every server kept its logs **only inside itself** – log in to app01 for the price script, log in to db01 for the database. A frozen or broken server, or deleted logs, and the answer was gone. So: **one central place for all logs – stor01.**

2 · The picture

app01 · journald: all lines **mixed**
sudo line | systemd line
price-update line (logger -t)
nginx line (after 5.4b)
`90-forward.conf` · omfwd · **queue waits here**

db01 · systemd, sshd, kernel …
+ PostgreSQL (`log_destination = 'stderr,syslog'`)
`90-forward.conf` · omfwd · queue

TCP 514
live, line by line
➜

stor01 · `10-receive.conf` · imtcp + omfile
/var/log/remote/
app01/ sudo.log · systemd.log · sshd.log · price-update.log · nginx_access.log · nginx_error.log
db01/ postgres.log · kernel.log · systemd.log …

folder = server name (automatic)
file = program name, or the tag from `logger -t` / nginx `tag=`

3 · Final paragraph

Getting lines into the system log – each program its own way. sudo, sshd, systemd and cron write to the system log by themselves. Our **script** adds its own lines with **logger** and a **tag** (`-t price-update`) – logger only *writes* a line, it does not collect anything; the tag lets `journalctl -t price-update` show just that script. Programs with **their own files** need one setting: PostgreSQL `log_destination = 'stderr,syslog'` (stderr = its normal print-out, which stays in its own file), **nginx** has two own files – the **access log** (`/var/log/nginx/access.log` , every page visit) and the **error log** (`/var/log/nginx/error.log` , problems) – so it needs **two** extra lines, one for each: `access_log syslog:server=unix:/dev/log,tag=nginx_access;` and `error_log syslog:server=unix:/dev/log,tag=nginx_error;` (its own files stay), Docker `"log-driver": "journald"` in `/etc/docker/daemon.json` (a setting you write yourself; default is json-file). A program with **no** such setting: rsyslog itself **watches its file** with **imfile** (see section 5).

Three copies of every line – two on app01, one on stor01. Copy 1 – journald on app01: binary, `/var/log/journal/` , read with `journalctl` . **Copy 2 – rsyslog's text copy on app01**: `/var/log/syslog` (and `/var/log/auth.log` for logins and sudo) – **not sorted per program**: almost all programs are mixed in one file. **Copy 3 – on stor01**: rsyslog (user **syslog**) **sends every line live, one by one** over TCP 514 (**omfwd**, `90-forward.conf` , `*.*`), with a **queue on the sender**; stor01 (**imtcp** receives, **omfile** writes, `10-receive.conf`) reads the **server name** and the **program name or tag** in each line and **sorts** it into `/var/log/remote/<server>/<program>.log` . So the **sorting per program happens only on stor01** – it is a real third copy, on another machine, and sorted: all sudo lines in `sudo.log` , all systemd lines in `systemd.log` . The names come from stor01's template, not from the forward file; `stop` keeps them out of stor01's own syslog.

Numbering. rsyslog reads `/etc/rsyslog.d/` alphabetically – `10-receive` first, Ubuntu's `50-default` , `90-forward` last. Order matters when one rule must act before another (rules after a `stop` never see those lines). The word after the number is only for people.

4 · Where each log is kept

Log	Where	Read with
journald (all system lines)	`/var/log/journal/<machine-id>/` on disk · if missing: `/run/log/journal/` in RAM, lost at restart (Step 5.5)	`journalctl` (binary)
rsyslog text copies	`/var/log/syslog` (almost all) · `/var/log/auth.log` (logins, sudo)	tail, less, grep
nginx (own files)	`/var/log/nginx/access.log` · `error.log`	tail, less
PostgreSQL 14 (own file)	`/var/log/postgresql/postgresql-14-main.log`	tail, less
Docker containers (default)	`/var/lib/docker/containers/<id>/<id>-json.log`	`docker logs <name>`
stor01 central copy	`/var/log/remote/<server>/<program>.log`	tail, less, grep

5 · Any program – how its lines reach stor01

Where the program writes	Examples	What to do
1 · the system log	sudo, sshd, systemd, cron, logger, any systemd service that prints text	nothing – journald → rsyslog → stor01
2a · own files, with a syslog setting	PostgreSQL, nginx	one setting – the program also writes to syslog; its own file stays
2b · own files, no setting	some apps that can only write their own file	rsyslog **watches the file itself** with **imfile** (**i**nput **m**odule **file**) – like `tail -f` : every new line written to the file is picked up, given a tag, and sent like any other line. One small drop-in on that server: `module(load="imfile")` `input(type="imfile" File="/var/log/myapp/app.log" Tag="myapp")` → on stor01: `/var/log/remote/app01/myapp.log`
3 · Docker	any container	`{ "log-driver": "journald" }` in `/etc/docker/daemon.json` + `sudo systemctl restart docker` (new containers only)

Rule to remember: first get the program's lines into the system log – each program its own way; then rsyslog sends them all the same way, as **lines**, not files.

6 · Users

Program	Runs as	Why it matters
rsyslog	**syslog**	`/var/log/remote` must belong to syslog, or it cannot write
PostgreSQL	**postgres**	owns the database files and its log
journald · sshd	**root**	keeps all logs · handles logins
nginx	**root** (main) + **www-data** (workers)	the workers that serve pages cannot change the system

7 · Standard setup – logs to stor01 from day one (Step 5.4b, planned)

Instead of setting up logging by hand for every program, each server gets a **standard setup**: one small **drop-in file** per program (no edit of the program's main file), placed by **one script**.

Program	Drop-in file	The setting inside
rsyslog (every server)	`/etc/rsyslog.d/90-forward.conf`	`*.*` → omfwd to stor01:514, with a queue
nginx	`/etc/nginx/conf.d/central-logs.conf`	`access_log syslog:server=unix:/dev/log,tag=nginx_access;` `error_log syslog:server=unix:/dev/log,tag=nginx_error;`
PostgreSQL	`/etc/postgresql/14/main/conf.d/central-logs.conf`	`log_destination = 'stderr,syslog'`
Docker	`/etc/docker/daemon.json`	`{ "log-driver": "journald" }`

Why drop-in files: an update of nginx or PostgreSQL does not overwrite them · to undo, delete one file · every server looks the same.

```
enable-central-logs.sh   (run once on any server, with sudo)
 ├─ always              → 90-forward.conf      → rsyslogd -N1 → restart rsyslog
 ├─ nginx installed?    → central-logs.conf    → nginx -t     → reload nginx
 ├─ PostgreSQL there?   → central-logs.conf    →              → reload postgresql
 └─ Docker installed?   → daemon.json          →              → restart docker
```

The script checks which programs are on the server (like `server-report.sh` does), places only the files that fit, tests them and reloads the programs. In companies the same job is done by an **Ansible role** – one command sets up 5 or 500 servers, and every new server gets it when it is built.

Final paragraph. Every program first has to get its lines into the system log, each in its own way – so we give each one a small drop-in file with its one setting: rsyslog `90-forward.conf` (send everything to stor01), nginx `conf.d/central-logs.conf` (`syslog:` lines), PostgreSQL `conf.d/central-logs.conf` (`log_destination`), Docker `daemon.json` (`log-driver`). One script, `enable-central-logs.sh`, checks what is installed, places the matching files, tests them and reloads the programs – so every server and every new program sends its logs to stor01 from day one. Later this becomes an Ansible role. **Status:** planned as Step 5.4b – not done in the lab yet.

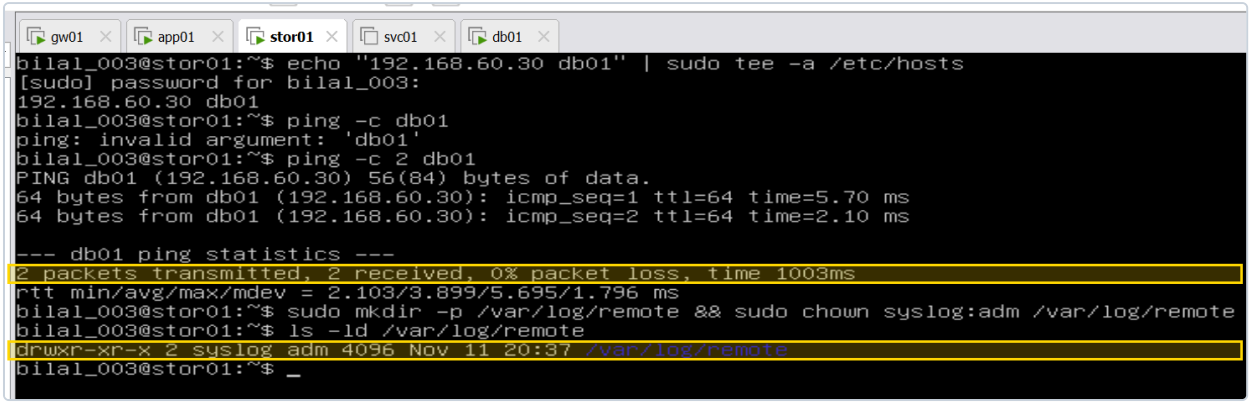
Part A · stor01 receives

A1 stor01 learns the name db01

stor01

Why (cause): stor01 will receive logs from db01 and must know who "db01" is. Without the name, ping db01 says "Name or service not known".

```
ssh bilal_003@stor01
echo "192.168.60.30 db01" | sudo tee -a /etc/hosts   # add one line to the names list
ping -c 2 db01                                      # 2 test packets
```



Look at (effect):

- **2 received, 0% packet loss** → stor01 reaches db01 by name.

If BAD: Destination Host Unreachable = db01 is switched off.

A2 A folder for the incoming logs

stor01

Why (cause): The logs from other servers need their own place. rsyslog on Ubuntu runs as the user **syslog**, not root – so the folder must belong to syslog, or rsyslog cannot write ("Permission denied").

```
sudo mkdir -p /var/log/remote
sudo chown syslog:adm /var/log/remote
ls -ld /var/log/remote
```

Look at (effect):

- Same screenshot as A1, last yellow line: **drwxr-xr-x 2 syslog adm … /var/log/remote** → the folder exists and belongs to syslog.

If BAD: root root in the line = chown was forgotten.

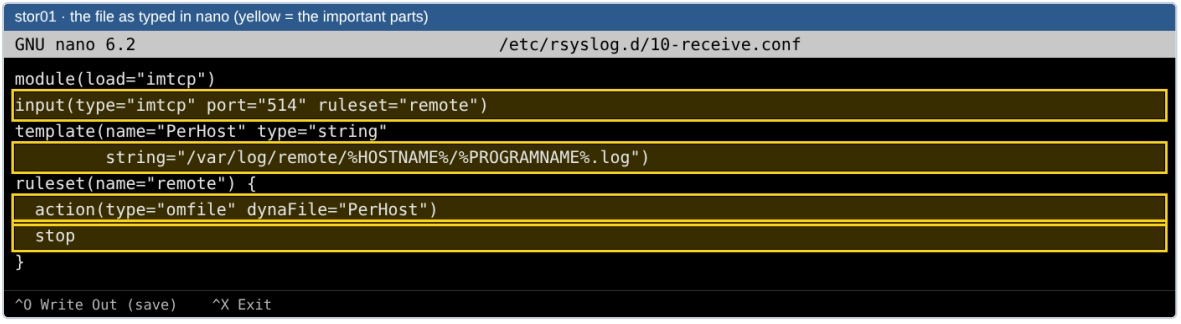
A3 The "receive" settings file

stor01

Why (cause): Tell rsyslog on stor01: listen on port 514, and save every line from another server into its own folder and file.

```
sudo nano /etc/rsyslog.d/10-receive.conf      # type these lines, save: Ctrl+O, Enter · exit: Ctrl+X

module(load="imtcp")                                    # im = input module: receive over TCP
input(type="imtcp" port="514" ruleset="remote")         # listen on 514, pass lines to "remote"
template(name="PerHost" type="string"
  string="/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log")  # one folder per server, one file per program
ruleset(name="remote") {
  action(type="omfile" dynaFile="PerHost")             # om = output module: write the line to that file
  stop                                                  # and nowhere else (not into stor01's own syslog)
}
```



Look at (effect):

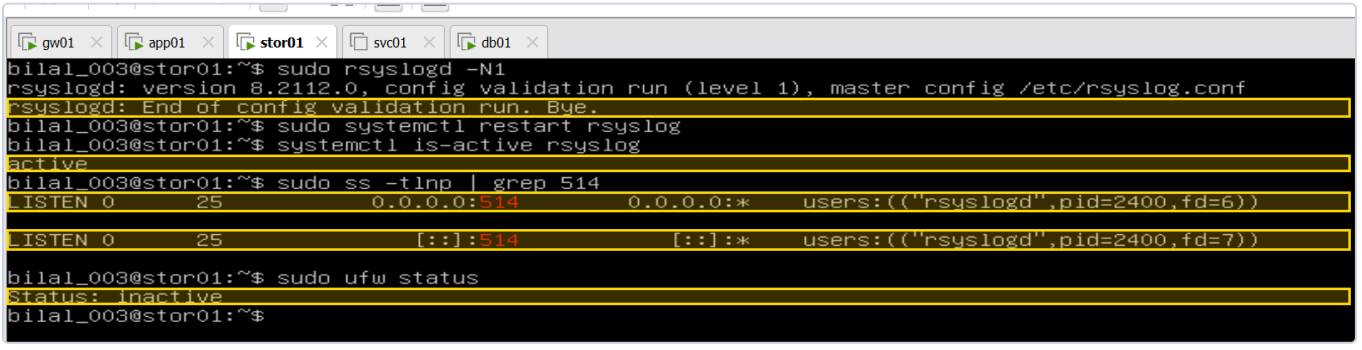
- **port 514** = the door · **string="/var/log/remote/…"** = where to save (starts with */*) · **omfile** = write to that file · **stop** = nowhere else.
- The folder **app01** and the file **ticket10.log** are not made by hand – the template makes them when the first line arrives.

A4 Check the file, restart, prove stor01 listens

stor01

Why (cause): Before a restart, check the file for typos. After the restart, prove that stor01 really listens on 514 and that no firewall blocks it.

```
sudo rsyslogd -N1                # check spelling of all rsyslog files
sudo systemctl restart rsyslog   # load the new file
systemctl is-active rsyslog
sudo ss -tlnp | grep 514         # who listens on port 514?
sudo ufw status                  # is a firewall on?
```



Look at (effect):

- **End of config validation run. Bye.** → no spelling errors.
- **active** → rsyslog runs with the new file.
- **LISTEN 0.0.0.0:514 … rsyslogd** (and [::]:514 for IPv6) → stor01 is waiting for logs.
- **Status: inactive** → no firewall, nothing to open. **Part A done.**

If BAD: An error with a line number after -N1 → fix that line. No 514 line → rsyslog did not load the file.

Part B · app01 sends

B1 The send rule on app01

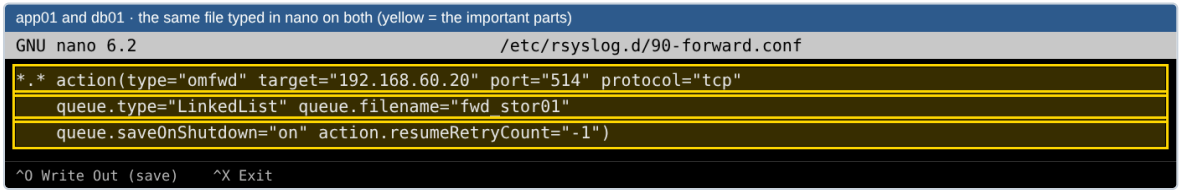
app01

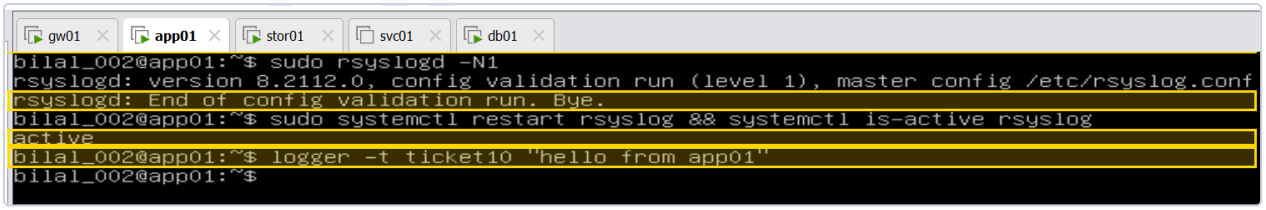
Why (cause): Send every log line of app01 to stor01 – and keep lines in a queue if stor01 cannot be reached. stor01 only receives, so this file exists only on app01 and db01.

```
sudo nano /etc/rsyslog.d/90-forward.conf

*.* action(type="omfwd" target="192.168.60.20" port="514" protocol="tcp"   # every line → stor01:514, TCP
    queue.type="LinkedList" queue.filename="fwd_stor01"                     # waiting line (queue), saved to
disk if needed
    queue.saveOnShutdown="on" action.resumeRetryCount="-1")                 # keep it over a restart · never
stop trying

# save: Ctrl+O, Enter · exit: Ctrl+X
sudo rsyslogd -N1
sudo systemctl restart rsyslog && systemctl is-active rsyslog
logger -t ticket10 "hello from app01"       # one test line with the tag ticket10
```





Look at (effect):

- The rule ends with **)** – it opens with (on line 1 and closes on line 3.
- **Bye.** + **active** → app01 now sends everything to stor01.
- **logger -t ticket10** → the test line. The tag becomes the file name on stor01.

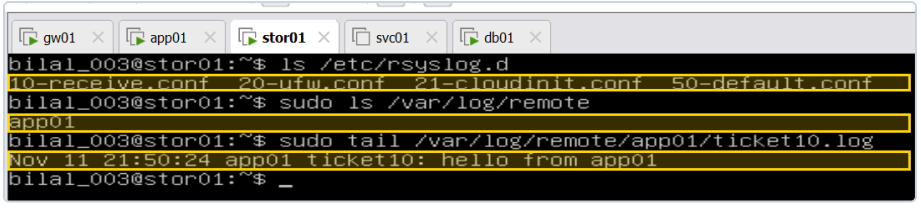
Part C · Tests – do the logs arrive? (and db01 joins)

C1 First proof – app01's line on stor01

stor01

Why (cause): Does the test line from app01 arrive, in the right folder and file?

```
ls /etc/rsyslog.d                                  # stor01 must have only the receive file
sudo ls /var/log/remote
sudo tail /var/log/remote/app01/ticket10.log
```



Look at (effect):

- **10-receive.conf**, no 90-forward.conf → stor01 only receives (20-ufw, 21-cloudinit, 50-default are Ubuntu's own).
- **app01** folder → created automatically by the template.
- **Nov 11 21:50:24 app01 ticket10: hello from app01** → written on app01, readable on stor01. Time in UTC (1 h behind German time).

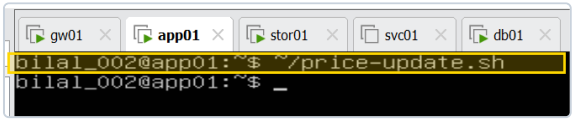
If BAD: No app01 folder → on app01: `sudo systemctl status rsyslog`, look for "suspended" / "connection refused".

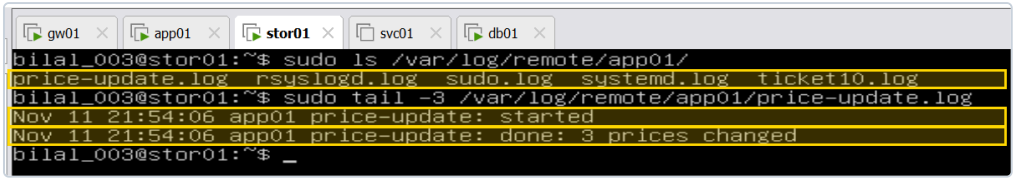
C2 Ticket #8 – the price script's diary reaches stor01

app01 → stor01

Why (cause): Ticket #8 gave the price script log lines with logger – but only on app01. Now they should also arrive on stor01.

```
~/price-update.sh                                          # app01 – runs silently
sudo ls /var/log/remote/app01/                             # stor01
sudo tail -3 /var/log/remote/app01/price-update.log        # stor01
```





Look at (effect):

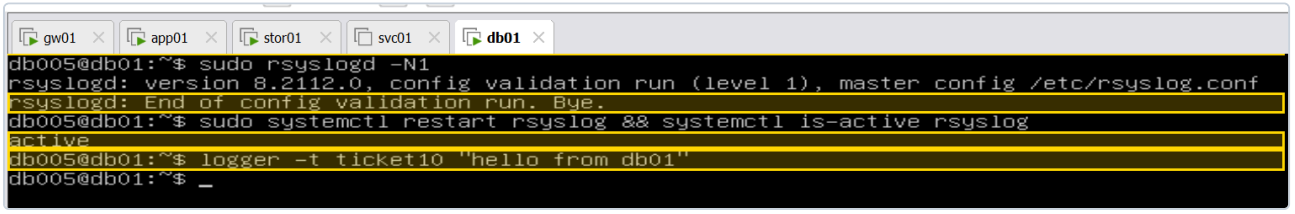
- **price-update.log** next to sudo.log, systemd.log … → app01 sends *everything* (*.*), each program gets its own file.
- **started** + **done: 3 prices changed** → Markus's question "did it run?" is answered on stor01, without logging in to app01.

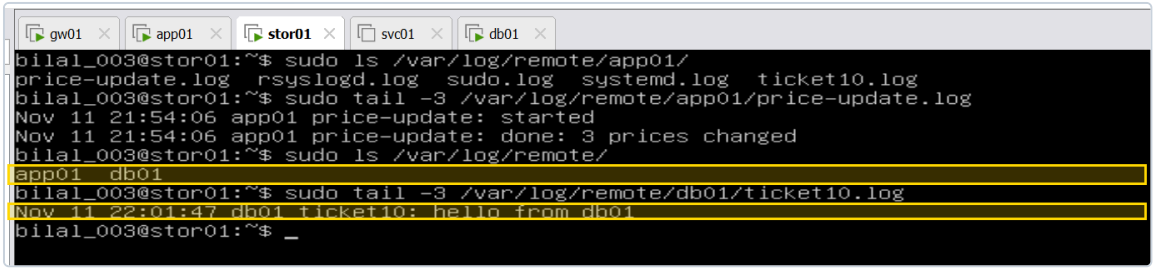
C3 db01 sends too

db01 → stor01

Why (cause): The database server needs the same send rule. db01 has no NFS share, so the file was typed there.

```
sudo nano /etc/rsyslog.d/90-forward.conf      # same 3 lines as app01 (see B1)
sudo rsyslogd -N1 && sudo systemctl restart rsyslog && systemctl is-active rsyslog
logger -t ticket10 "hello from db01"
sudo ls /var/log/remote/ ; sudo tail -3 /var/log/remote/db01/ticket10.log     # stor01
```





Look at (effect):

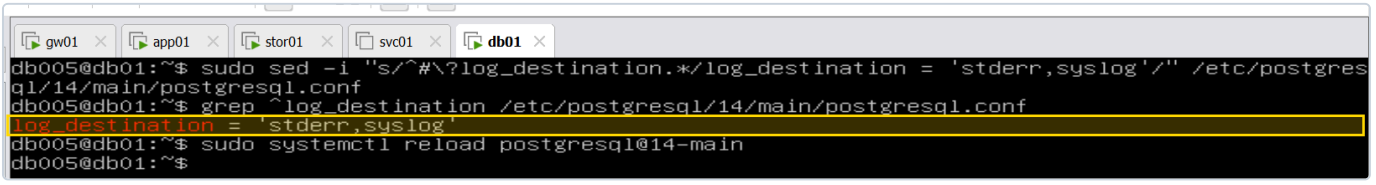
- db01: **Bye.** · **active** · logger.
- stor01: **app01 db01** → two servers send · **db01 ticket10: hello from db01**.

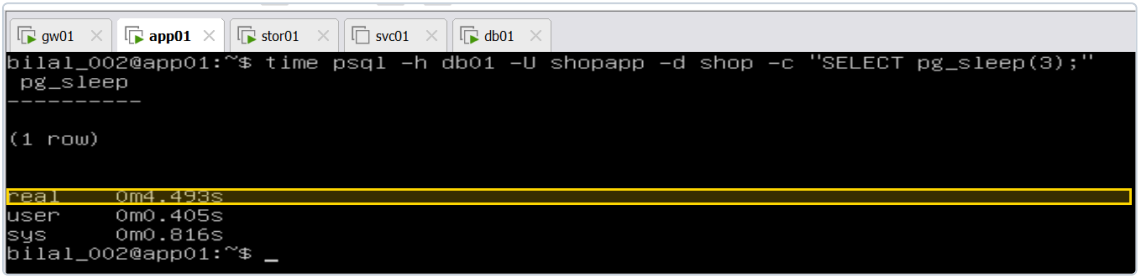
C4 Ticket #9 – the slow-query line reaches stor01

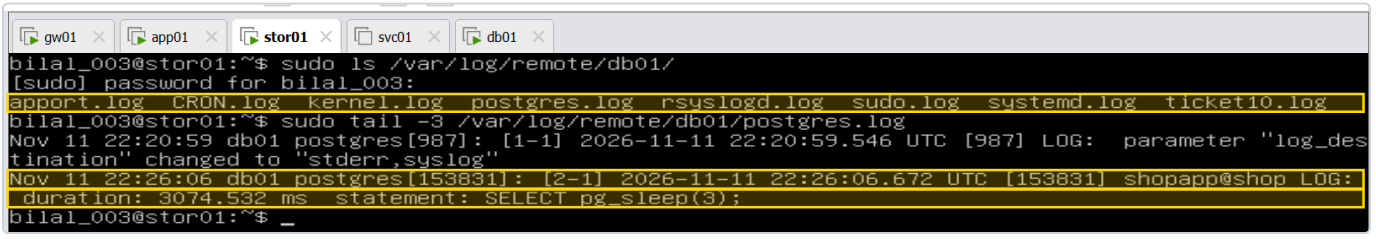
db01 · app01 · stor01

Why (cause): PostgreSQL on Ubuntu writes to **its own file** (/var/log/postgresql/…), not to rsyslog – so its lines did not reach stor01. One setting adds rsyslog as a second place. **reload** = read settings again without stopping the database (the shop keeps running).

```
sudo sed -i "s/^#\?log_destination.*/log_destination = 'stderr,syslog'/"
/etc/postgresql/14/main/postgresql.conf   # db01
grep ^log_destination /etc/postgresql/14/main/postgresql.conf
# db01
sudo systemctl reload postgresql@14-main
# db01
time psql -h db01 -U shopapp -d shop -c "SELECT pg_sleep(3);"     # app01 – a 3 s query (limit is 2 s)
sudo tail -3 /var/log/remote/db01/postgres.log                    # stor01
```







Look at (effect):

- **log_destination = 'stderr,syslog'** → own file *and* rsyslog.
- **real 0m4.493s** → 3 s sleep + connecting and login.
- **postgres.log** and **kernel.log** in db01's folder · **duration: 3074.532 ms statement: SELECT pg_sleep(3);** → ticket #9 case ③ evidence on stor01, without logging in to db01. A "CPU stuck" line (case ②) would land in kernel.log.

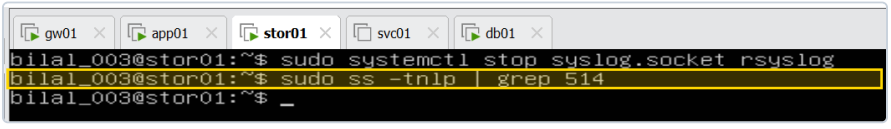
Part D · Failure test – stor01 down for a while

D1 stor01 "goes down"

stor01

Why (cause): Prove the queue: what happens to app01's logs while stor01 is down? First switch off the receiver (syslog.socket too, so Ubuntu does not start rsyslog again by itself).

```
sudo systemctl stop syslog.socket rsyslog
sudo ss -tlnp | grep 514
```



Look at (effect):

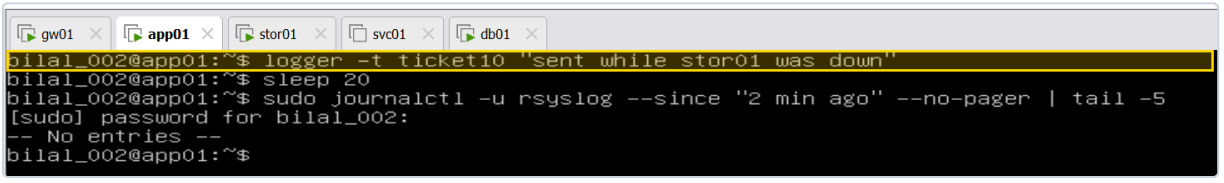
- **No output after ss** → port 514 is closed. This is the outage.

D2 app01 logs during the outage

app01

Why (cause): app01 writes a line it cannot deliver. rsyslog keeps it in the queue fwd_stor01 instead of throwing it away.

```
logger -t ticket10 "sent while stor01 was down"
sleep 20
sudo journalctl -u rsyslog --since "2 min ago" --no-pager | tail -5
```



Look at (effect):

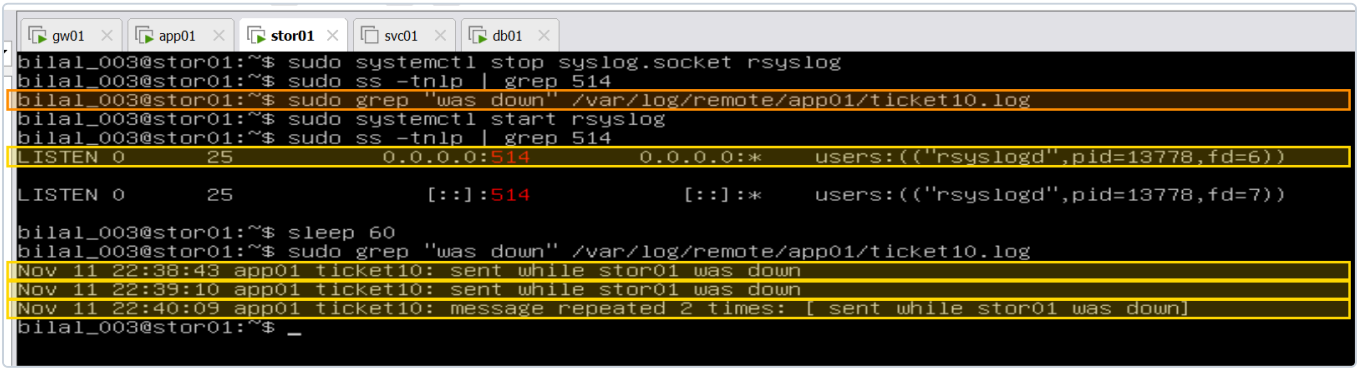
- **logger** → the line is written on app01.
- "-- No entries --" → rsyslog did not report the outage in the journal – normal, ignore for this ticket.

D3 stor01 back – the waiting line arrives

stor01

Why (cause): Before: the line must NOT be on stor01. After the start: app01 retries (about every 30 s) and sends the waiting line.

```
sudo grep "was down" /var/log/remote/app01/ticket10.log     # before – nothing
sudo systemctl start rsyslog
sudo ss -tlnp | grep 514
sleep 60
sudo grep "was down" /var/log/remote/app01/ticket10.log     # after – the line is there
```



Look at (effect):

- **Orange:** the first grep → no output → the line was not on stor01 during the outage.
- **LISTEN 0.0.0.0:514** → stor01 is back.
- **22:38:43 / 22:39:10 … sent while stor01 was down** → the lines arrived with the time they were *written*, not the time they arrived. **No line lost.**
- **message repeated 2 times** → the same logger line was run again; rsyslog folds identical lines into one.

If BAD: Still nothing after 60 s → wait another minute (the retry gap grows) and grep again.

Result

GLPI · Assistance › Tickets › #10

Title	**Logs are spread over every server – after a freeze we cannot read them**
Status	**Solved**
Requester	Sara (IT lead) · Category: Linux › Logging · Priority: High
Servers	app01, db01 (send) · stor01 (central log server)
Opened / solved	11 Nov 2026 · same day

✔ Solution

What was done: stor01 is now the central log server. rsyslog on stor01 receives on TCP port 514 and keeps every line in `/var/log/remote/<server>/<program>.log`. app01 and db01 send every log line there, with a queue on their own disk. PostgreSQL on db01 now writes to rsyslog too (`log_destination = 'stderr,syslog'`, reload without downtime).

Proof:
• app01: `logger` test line and the Black Friday price script (ticket #8: started / done: 3 prices changed) are readable on stor01.
• db01: test line and the slow-query line from ticket #9 (`duration: 3074 ms statement: SELECT pg_sleep(3)`) are readable on stor01.
• Failure test: rsyslog on stor01 stopped → app01 logged "sent while stor01 was down" → the line was not on stor01 → stor01 started again → the line arrived, with its original time. **No log line lost.**

For you: logs of app01 and db01 can now be read on stor01 – also when a server is frozen, broken, or someone deleted its logs (attacker, insider or by mistake). Who deleted or changed a log file will be recorded with auditd in Phase 6.

Final paragraph: stor01 is the central log server – rsyslog listens on TCP 514 and writes every line into /var/log/remote/<server>/<program>.log (folder owned by syslog, path starting with /). app01 and db01 send every line to 192.168.60.20:514 with a queue; the receiver has only the receive file, the senders only the send file. On stor01, without logging in anywhere else: the test lines of app01 and db01, the price script's started / done lines (ticket #8) and PostgreSQL's slow-query line (ticket #9, after log_destination = 'stderr,syslog' and a reload). With stor01's rsyslog stopped, app01 kept its new line in the queue and sent it as soon as stor01 was back, with its original time – no line lost.

Next: Step 5.5 persistent journal (each server keeps its own log on disk across restarts) · Step 5.6 logrotate (so /var/log/remote cannot fill stor01's disk) · Phase 6 auditd (*who* changed or deleted a log file).